ROTEIRO PRÁTICO DOC • PROGRAMAÇÃO PARA A INTERNET 2

Arquitetura Modular do Frontend

Documentação técnica da estrutura em camadas com ES6 Modules e desacoplamento do DOM

Arquitetura Web · ES6 Modules · Storage & API · IFSC Garopaba

Unidade Curricular
Programação para a Internet 2
CST em Sistemas para a Internet

Docente
Prof. André Moraes
IFSC Câmpus Garopaba

Produto deste roteiro
Registro da arquitetura modular do portal, guia de manutenção e referência pedagógica viva

Ao final desta prática você será capaz de...

- **Registrar formalmente a arquitetura modular** adotada no portal didático da disciplina de Programação para a Internet 2;
- **Descrever a divisão em quatro camadas coesas** (`config/`, `utils/`, `services/` e `views/`), alinhadas aos conteúdos ensinados no Roteiro 04;
- **Documentar a eliminação do monólito de 5.445 linhas de script inline** do `index.html`, demonstrando ganho de performance e manutenibilidade;
- **Explicar a ponte de retrocompatibilidade global** estabelecida via `app.js` para elementos declarativos do DOM legados;
- **Fornecer um guia cirúrgico para novas funcionalidades**, permitindo adicionar demandas, serviços e rotas com isolamento e segurança;
- **Apresentar a matriz de métricas e testes de regressão** que asseguram a estabilidade do portal no GitHub Pages.

Mapa da aula

1 Contexto & Motivação
Gargalos do monólito e alinhamento pedagógico (Seção 1)

2 Visão Geral das Camadas
Diagrama de fluxo, contratos e responsabilidades (Seção 2)

3 Estrutura de Arquivos
Módulos em `js/`, serviços e ponte com o DOM (Seção 3)

4 Guia de Extensão & Métricas
Como estender o portal e matriz de impacto (Seção 4)

1 Contexto e Motivação da Refatoração

Durante o desenvolvimento inicial do portal unificado da disciplina de Programação para a Internet 2, o arquivo `index.html` concentrava tanto a marcação semântica e os modais Bootstrap 5 quanto a totalidade da lógica em JavaScript da aplicação. Tratava-se de um monólito com **mais de 5.445 linhas de código inline** contidas em uma única tag `<script>`, elevando o tamanho do documento para impressionantes 11.070 linhas.

> **A Importância do Desacoplamento Pedagógico**
>
> Em sala de aula, ensina-se aos futuros tecnólogos a separação estrita de interesses (*Separation of Concerns*). A prática pedagógica é consolidada no **Roteiro Prático 04 — Arquitetura Modular com ES6 Modules**, no qual os estudantes estruturam aplicações web separando utilitários, serviços de persistência e camadas de visualização. Manter o portal da própria unidade curricular em formato monolítico criava uma contradição didática que a presente refatoração eliminou integralmente.

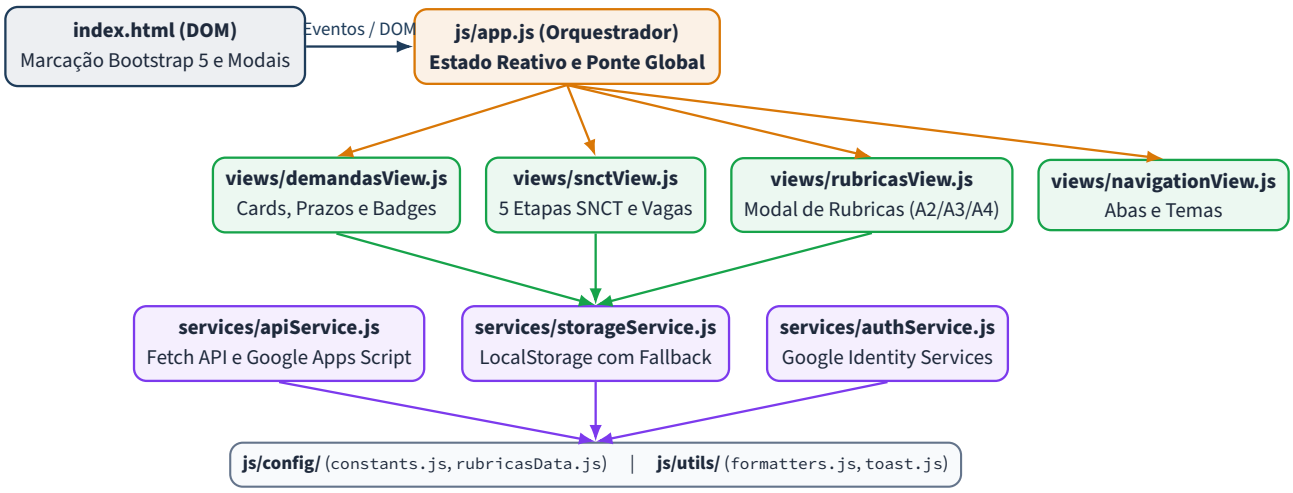
Esse cenário anterior gerava três ordens de desafios críticos:

1. **Sobrecarga de Contexto e Degradação de Manutenção:** A adição ou correção de qualquer elemento visual (como filtros da Central de Demandas ou cartões da SNCT) exigia a inspeção de milhares de linhas, aumentando a latência das intervenções e o risco de colisões;
2. **Acoplamento Severo com o DOM e Armazenamento:** Constantes globais, manipulação direta de `localStorage`, rotinas de OAuth2 com Google Identity Services e geradores de templates HTML compartilhavam o mesmo escopo global sem limites arquiteturais;
3. **Dificuldade na Adição de Testes e Evolução:** Funções utilitárias como sanitização de strings contra XSS (`escapeHtml`) e formatação temporal ficavam inacessíveis para reuso isolado.

Com a refatoração concluída em 4 fases cirúrgicas, o `index.html` foi reduzido em **5.448 linhas (corte de 49,2%)**, tornando-se um arquivo estritamente declarativo, enquanto toda a inteligência da aplicação passou a operar via módulos nativos do ES6 (`<scripttype="module"src="js/app.js">`).

2 Visão Geral da Arquitetura em Camadas

A aplicação foi estruturada seguindo o padrão de camadas unidirecionais, onde a apresentação depende dos serviços e configurações, mas as camadas inferiores jamais conhecem detalhes de interface:



Camada	Responsabilidade Arquitetural	Principais Módulos
Apresentação (`views/`)	Geração declarativa de HTML dinâmico, atualização de badges, gerenciamento de modais e captura de ações do usuário.	`demandasView.js`, `snctView.js`, `rubricasView.js`, `navigationView.js`
Serviços (`services/`)	Isolamento de operações de entrada e saída (I/O), persistência local, chamadas HTTP remotas e autenticação OAuth2.	`storageService.js`, `apiService.js`, `authService.js`
Utilitários (`utils/`)	Funções puras sem efeitos colaterais para sanitização contra XSS, conversão de datas e toasts flutuantes.	`formatters.js`, `toast.js`
Configuração (`config/`)	Centralização de chaves de storage, roster dos 24 alunos, frentes da SNCT e matrizes pedagógicas formativas.	`constants.js`, `rubricasData.js`
Orquestrador (`app.js`)	Ponto de entrada modular, gerenciador do estado em memória e ponte de retrocompatibilidade global para o HTML.	`app.js`, `main.js`

Tabela 1: Matriz de responsabilidades das camadas da aplicação.

3 Estrutura de Diretórios e Contratos de Módulos

A árvore de arquivos do portal reflete de forma fidedigna as diretrizes de engenharia de software frontend modernas:

Estrutura Modular do Portal da UC ProgWeb 2

```
uc-programacao-para-a-internet-2/
|-- index.html                # Dashboard da disciplina (sem blocos <script> inline)
|-- progweb2-config.js        # Parametrização global de ambiente (URLs e Client ID)
`-- js/
    |-- app.js                # Ponto de entrada carregado via <script type="module">
    |-- main.js               # Arquivo de barramento e re-exportação de contratos
    |-- config/
    |   |-- constants.js      # Chaves de armazenamento, roster de 24 alunos e frentes SNCT
    |   `-- rubricasData.js   # Matrizes contextuais de rubricas formativas (A2, A3 e A4)
    |-- services/
    |   |-- apiService.js     # Integração assíncrona com backend Google Apps Script
    |   |-- authService.js    # Autenticação Google GIS, decodificação JWT e vinculação
    |   `-- storageService.js # Operações de CRUD com LocalStorage e fallback tolerante
    |-- utils/
    |   |-- formatters.js     # Sanitização XSS, formatação de datas e cálculo de prazos
    |   `-- toast.js          # Notificações visuais flutuantes com Bootstrap Toasts
    `-- views/
        |-- demandasView.js   # Renderização da Central de Demandas, cards e filtros
        |-- navigationView.js # Chaveamento de abas, tema claro/escuro e foco de card
        |-- rubricasView.js   # Painel interativo de rubricas e gerador de parecer
        `-- snctView.js       # Diagrama das 5 etapas da SNCT e quadro de vagas
```

3.1 Camada de Configuração (`js/config/`)

O módulo `constants.js` concentra todos os literais imutáveis do portal. Nenhuma chave de armazenamento é escrita como texto solto no código-fonte:

`js/config/constants.js` — JavaScript

```javascript
export const STORAGE_KEY_DEMANDAS = "progweb2_demandas_v1";
export const STORAGE_KEY_AVALIACOES = "progweb2_avaliacoes_v1";
export const STORAGE_KEY_ALUNOS = "progweb2_alunos_v1";
export const STORAGE_KEY_SNCT_DEMANDAS = "progweb2_snct_demandas_v1";
export const STORAGE_KEY_GOOGLE_SESSION = "progweb2_google_session_v1";

// Roster oficial completo com os 24 estudantes da turma 2026/2
export const ESTUDANTES_PADRAO = [
  { id: "aluno_1", nome: "Alexandre Silva", matricula: "2024101001", email: "alexandre@aluno.ifsc.edu.br"
    },
  { id: "aluno_2", nome: "Beatriz Oliveira", matricula: "2024101002", email: "beatriz@aluno.ifsc.edu.br"
    },
  // ... 22 demais discentes devidamente catalogados
];
```

Já o arquivo `rubricasData.js` formaliza as três matrizes pedagógicas adotadas nas avaliações formativas da unidade curricular:

- **Matriz A2 (SNCT):** Focada em planejamento didático, rigor conceitual da oficina, ambientação na sala

Moodle e trabalho em equipe;

- **Matriz A3 (Projeto Integrador):** Focada em backend RESTful com Flask/SQLite, integração com Fetch API, contingência offline e ergonomia visual;
- **Matriz A4 (Roteiros Práticos):** Focada em domínio de ES6 Modules, manipulação do DOM sem bibliotecas pesadas, formulários e responsividade.

3.2 Camada de Serviços e I/O (`services/`)

A persistência no navegador é completamente intermediada por `storageService.js`. Caso o navegador esteja em modo anônimo estrito ou com cotas esgotadas, o serviço captura as exceções e mantém a operação estável em memória:

js/services/storageService.js — JavaScript

```javascript
import { STORAGE_KEY_DEMANDAS, DEMANDAS_INICIAIS } from "../config/constants.js";

export function carregarDemandas() {
  try {
    const raw = localStorage.getItem(STORAGE_KEY_DEMANDAS);
    if (!raw) return DEMANDAS_INICIAIS;
    const lista = JSON.parse(raw);
    return Array.isArray(lista) && lista.length > 0 ? lista : DEMANDAS_INICIAIS;
  } catch (erro) {
    console.warn("Falha ao acessar LocalStorage; utilizando dados de contingência:", erro);
    return DEMANDAS_INICIAIS;
  }
}

export function salvarDemandas(demandas) {
  try {
    localStorage.setItem(STORAGE_KEY_DEMANDAS, JSON.stringify(demandas));
    return true;
  } catch (erro) {
    console.error("Erro crítico ao gravar demandas no LocalStorage:", erro);
    return false;
  }
}
```

A comunicação externa com a planilha Google Sheets via Google Apps Script é centralizada em `apiService.js`. O método utiliza cabeçalho `text/plain;charset=utf-8` para evitar bloqueios de pré-voo CORS (*CORS preflight*):

js/services/apiService.js — JavaScript

```javascript
export async function chamarApiAppsScript(acao, payload = {}) {
  const url = window.PROGWEB2_CONFIG?.appsScriptUrl;
  if (!url) throw new Error("URL da API do Google Apps Script não configurada.");

  const corpoRequisicao = JSON.stringify({ acao, ...payload, timestamp: new Date().toISOString() });
```

```
  const resposta = await fetch(url, {
    method: "POST",
    headers: { "Content-Type": "text/plain;charset=utf-8" },
    body: corpoRequisicao
  });

  if (!resposta.ok) throw new Error(`Falha na requisição HTTP: status ${resposta.status}`);
  return await resposta.json();
}
```

3.3 A Ponte de Retrocompatibilidade no `js/app.js`

Uma das maiores inovações arquiteturais desta refatoração foi garantir que o `index.html` continuasse funcionando sem a necessidade de reescrever centenas de atributos de manipuladores declarativos legados (`onclick="..."` e `onchange="..."`).

> **Escopo de Módulos vs. Escopo Global (`window`)**
>
> Ao carregar um script com `type="module"`, o navegador executa o código em escopo fechado. As funções definidas dentro do módulo **não existem no objeto global `window`**. Para viabilizar a transição sem quebras no HTML existente, o `js/app.js` expõe explicitamente as 79 funções manipuladoras ao escopo global.

js/app.js — JavaScript

```
// Vinculação controlada das rotinas públicas ao objeto global window
const manipuladoresGlobais = {
  showTab, toggleTheme, alternarDestaqueCard,
  filtrarDemandas, abrirModalNovaDemanda, salvarDemandaFormulario,
  navegarParaEtapaSNCT, abrirModalInscricaoSNCT, confirmarInscricaoSNCT,
  abrirModalAvaliacaoRubrica, salvarAvaliacaoRubrica,
  iniciarLoginGoogle, efetuarLogoutGoogle, sincronizarComBackendAppsScript
  // ... mapeamento das 79 rotinas públicas necessárias
};

Object.entries(manipuladoresGlobais).forEach(([nome, funcao]) => {
  if (typeof funcao === "function") {
    window[nome] = funcao;
  }
});
```

4 Guia de Manutenção & Métricas de Impacto

4.1 Como Adicionar uma Nova Funcionalidade ao Portal

Para manter a integridade da arquitetura em camadas nas próximas manutenções, siga rigorosamente o fluxo de desenvolvimento abaixo:

1. **Novas Constantes ou Roster:** Caso surjam novas turmas, novas chaves de cache ou novos instrumentos avaliativos, registre-os em `js/config/constants.js` ou `js/config/rubricasData.js`;
2. **Acesso a Dados:** Qualquer necessidade de consulta ou escrita em `localStorage` ou chamadas remotas de rede deve receber um método correspondente em `js/services/`;
3. **Renderização de Componentes:** A construção de strings HTML de cards ou tabelas pertence exclusivamente aos módulos da pasta `js/views/`. Use sempre `escapeHtml()` para qualquer dado digitado por usuário ou estudante;
4. **Ponto de Conexão no DOM:** Se o novo componente demandar interação direta via atributo HTML (`onclick`), inclua a função correspondente no mapa de vinculação em `js/app.js`.

> **Regra de Ouro da Camada de Visualização**
>
> Nenhum módulo da pasta `js/views/` deve executar diretamente comandos como `localStorage.setItem()` ou invocar `fetch()` para serviços externos. Toda a persistência deve obrigatoriamente transitar por `js/services/storageService.js` ou `js/services/apiService.js`.

4.2 Matriz Comparativa de Métricas

A tabela a seguir demonstra quantitativamente o ganho obtido com a refatoração modular:

Métrica de Qualidade	Antes da Refatoração (Monólito)	Após a Refatoração (ES6 Modules)
Linhas totais no `index.html`	11.070 linhas	5.622 linhas (Redução de 49,2%)
Linhas de JavaScript inline no HTML	5.445 linhas em bloco único	0 linhas (100% desacoplado em módulos)
Organização do código	Escopo global desprotegido	11 arquivos coesos distribuídos em 4 camadas
Localização e isolamento de bugs	Varredura longa e propensa a erros	Diagnóstico pontual no módulo responsável
Aderência pedagógica	Divergente da ementa lecionada	Idêntica à arquitetura ensinada no Roteiro 04
Testabilidade de rotinas	Impossível isolar regras de negócio	Funções puras exportáveis e testáveis individualmente

Tabela 2: Comparativo das métricas de engenharia de software antes e após a modularização.

5 Checklist de Verificação Arquitetural

Checklist para validação de novas contribuições no portal

- [ ] **Zero scripts inline no HTML:** Nenhuma nova tag `<script>` com lógica de negócio foi adicionada ao `index.html`.
- [ ] **Desacoplamento do storage:** Nenhuma view executa leitura ou escrita direta no `localStorage`.
- [ ] **Sanitização contra XSS:** Todo dado dinâmico renderizado em template literals utiliza a função `escapeHtml()`.
- [ ] **Importações explícitas:** Todas as dependências entre arquivos utilizam caminhos relativos com extensão `.js`.
- [ ] **Ponte global registrada:** Funções chamadas por manipuladores declarativos no HTML foram exportadas em `js/app.js`.
- [ ] **Validação no GitHub Pages:** O portal carrega sem erros de sintaxe ou de CORS no console do navegador em produção.

Referência nos Roteiros da Disciplina

Para entender os fundamentos de importação e exportação de módulos, consulte o material de apoio em `roteiro-4/` (*Arquitetura Modular com ES6 Modules*), e para os detalhes da integração com APIs remotas, consulte `roteiro-5/` (*Fetch API & Flask*).